Detailed project walkthrough

This guide explains every project file and how the parts work together. Read the source files alongside the explanations. The key components summary gives a shorter overview, and the README contains deployment commands.

Contents

1. What the solution does
2. Project structure
3. AWS resources and access
4. Target configuration
5. The deployment and audit runner
6. Ansible configuration, inventory and playbook
7. The CloudWatch Agent role
8. Metrics, templates and alarms
9. The coverage auditor
10. Generated controller state
11. Dependencies and excluded files
12. Tests and verification
13. Complete flows and operational boundaries
14. Suggested reading order

1. What the solution does

The provider is AWS, and the VMs are EC2 instances. The solution monitors filesystem capacity on supported Linux and Windows hosts across configured accounts and regions.

There are three related activities:

Activity	Responsible component	Result
Set up monitoring	Ansible, started by the Python runner	Agents, configuration, disk expectations and alarms
Monitor disk capacity continuously	CloudWatch Agent and CloudWatch alarms	Metrics and low-space notifications
Verify monitoring coverage	Python coverage auditor	Findings about missing metrics, disks, connectivity or alarm configuration

Ansible is the configuration management tool. The CloudWatch Agent runs inside each VM and collects filesystem information. CloudWatch stores metrics and evaluates alarms even when the controller is unavailable. This makes continuous capacity monitoring independent of the deployment schedule.

A disk-capacity alarm means a disk has crossed its configured utilization threshold. A coverage finding means some part of monitoring cannot be verified. For example, a stopped agent may produce a coverage finding even if the disk has plenty of free space.

Terms used in this guide

Term	Meaning here
Central account	AWS account containing the monitoring hub and normally the controller
Workload or member account	AWS account containing the VMs being monitored
Target	One configured workload account and region
Controller	Linux machine running Ansible and the Python scripts
Inventory	List of VMs discovered by Ansible
Filesystem	Mounted storage such as Linux `/data` or Windows `C:`
Manifest	Local JSON file recording which filesystems a VM should have monitored
Metric	Timestamped numerical measurement, such as disk used percentage
Dimension	Name/value label identifying a particular metric series
Namespace	CloudWatch grouping for metrics; this project uses `CWAgent`
ARN	AWS resource identifier containing information such as service, account and region
OAM	Observability Access Manager, used for cross-account metric visibility

2. Project structure

```
.
├── README.md
├── targets.yml
├── requirements.txt
├── requirements-dev.txt
├── .gitignore
├── cloudformation/
│   ├── 01_central_monitoring_sink.yml
│   ├── 02_member_account_baseline.yml
│   ├── 03_member_region.yml
│   └── controller_policy.example.json
├── ansible/
│   ├── ansible.cfg
│   ├── requirements.yml
│   ├── playbook_deploy_monitoring.yml
│   ├── inventory/
│   │   ├── aws_ec2.yml
│   │   └── group_vars/all.yml
│   └── roles/cloudwatch_disk_agent/
│       ├── vars/main.yml
│       ├── tasks/
│       │   ├── main.yml
│       │   ├── discover.yml
│       │   ├── linux.yml
│       │   ├── windows.yml
│       │   └── alarms.yml
│       ├── handlers/main.yml
│       └── templates/
│           ├── amazon-cloudwatch-agent-linux.json.j2
│           └── amazon-cloudwatch-agent-windows.json.j2
├── scripts/
│   ├── run_monitoring.py
│   └── coverage_reconciler.py
├── tests/
│   ├── test_runner.py
│   ├── test_coverage_reconciler.py
│   └── simulate_disk_pressure.sh
└── docs/
    ├── key-components.md
    └── project-walkthrough.md
```

CloudFormation describes infrastructure and permissions. Ansible describes changes on VMs and alarm creation. Python coordinates account/region runs and independently checks coverage. Tests verify important failure cases.

The README is the practical deployment guide and contains the architecture diagram. `docs/key-components.md` is the executive component architecture summary. This file is the detailed explanation. Generated state and local dependency directories are excluded from the source tree above.

Small syntax details used throughout the files

Syntax	Meaning
YAML `key: value`	A named setting and its value
YAML indentation	Groups settings under their parent
YAML `-`	An item in a list
YAML `---`	Starts a YAML document
YAML `#`	A comment
YAML `|`	Multiline text that preserves line breaks
YAML `>-`	Multiline text folded into a string, without a trailing newline
Jinja `{{ variable }}`	Insert or calculate an Ansible value
Jinja `value | filter`	Transform a value, for example sorting or JSON encoding
`.j2` extension	Template rendered into a final file by Ansible

Quoted account IDs are strings. Quoting avoids treating identifiers as numbers. A string containing `${aws:InstanceId}` in an agent template is resolved by the agent; Ansible's template expressions use different delimiters.

3. AWS resources and access

3.1 Where resources live

Location	Resources
Central account	Existing controller IAM role
Central account, each monitored region	OAM sink, SNS topic, email subscription and dashboard
Each workload account	Execution IAM role, monitoring VM role and instance profile
Each workload account and region	OAM link and private S3 staging bucket
Each workload VM	SSM Agent and CloudWatch Agent
Each workload account and region	Published metrics and disk alarms

The IAM baseline is deployed once per account because IAM roles are account-wide. Regional resources are deployed for each account/region target. The controller can process several regions; it does not need a separate VM for each region.

3.2 `01_central_monitoring_sink.yml`

Source: central monitoring template.

This template creates the central hub in one region.

Its parameters are:

Parameter	Purpose
`OrganizationId`	Organization whose member accounts may share metrics and send disk-alarm notifications
`AcquiredAccountIds`	Optional comma-separated account IDs outside that Organization
`NotificationEmail`	Email recipient for the SNS subscription
`CentralControllerRoleArn`	Existing controller role permitted to publish coverage reports

`AllowedPattern` validates the expected format of IDs and ARNs. `HasAcquiredAccounts` tests whether the optional account list is present. Conditional statements are omitted when it is empty.

The resources are:

- **`CentralObservabilitySink`:** An OAM sink allowing approved accounts to create/update metric-sharing links. The Organization statement checks `aws:PrincipalOrgID`. The optional statement allows the explicitly supplied standalone accounts. The resource-type condition permits CloudWatch metrics.
- **`CentralDiskAlertsTopic`:** An SNS topic named `enterprise-disk-alerts`, with an email subscription. The recipient must confirm the subscription.
- **`CentralDiskAlertsTopicPolicy`:** Allows CloudWatch to publish notifications from approved sources and regional `Disk-*` alarms. The controller role has a separate publishing statement for coverage reports.
- **`CentralDiskDashboard`:** Two widgets search for Linux used-percent metrics and Windows free-percent metrics, with warning and critical annotation lines.

The SNS Organization condition uses `aws:SourceOrgID` because the publisher is the CloudWatch service acting for a source account. The OAM condition uses `aws:PrincipalOrgID` because it authorizes the account principal creating a link. The optional SNS statement checks `aws:SourceAccount` for acquired accounts.

The dashboard name includes the region because dashboard names are account-wide. Each widget queries its configured region. Dashboard threshold lines are fixed in the template; change them if the alarm thresholds change.

Outputs provide the sink ARN, topic ARN and dashboard name. The sink ARN is passed to member regional stacks, and the topic ARN goes into target configuration.

3.3 `02_member_account_baseline.yml`

Source: member IAM template.

This template creates three resources in a workload account:

1. **`EC2MonitoringInstanceRole`:** An EC2 role with `AmazonSSMManagedInstanceCore` and `CloudWatchAgentServerPolicy` managed policies. These support the two agents.
2. **`EC2MonitoringInstanceProfile`:** The container used to attach that IAM role to an EC2 instance.
3. **`AnsibleCrossAccountExecutionRole`:** The role assumed by the controller. Its trust policy names the exact central controller role ARN.

The VM role and execution role serve different users: the VM agents use the first, while the controller uses the second. Do not replace an existing application role with the monitoring role; add the required permissions to the existing role instead.

The execution role's policy statements are:

Statement	Allowed work	Scope
`DiscoverInstancesAndMetrics`	Describe EC2/SSM state, list metrics, read metric data and describe alarms	Read APIs using `*` where resource-level restrictions do not apply
`ConfigureDiskAlarms`	Create/update alarms and enable their actions	Workload-account alarm ARNs beginning `Disk-`
`StartShellSessions`	Start SSM sessions	Account EC2 instances and the default shell-session document
`OpenSessionDataChannel`	Open the Session Manager data channel	`*`, as this action lacks resource-level permissions
`EndOwnSessions`	Terminate sessions belonging to the current caller	Session ARNs with the caller's session-ID tag condition
`LocateStagingBuckets`	List/locate staging buckets	Buckets matching this account's staging-bucket prefix
`TransferAndRemoveModules`	Upload, download and delete staged objects	Objects inside those buckets

`cloudwatch:ListMetrics` is granted, but the current auditor checks expected series through `GetMetricData`. Runtime permissions do not include alarm deletion or infrastructure provisioning.

Outputs expose the execution role ARN, VM instance-profile name and VM role ARN.

3.4 `03_member_region.yml`

Source: member regional template.

This template accepts the central sink ARN from the same region and creates:

- **`MemberObservabilityLink`:** Shares only CloudWatch metrics from the `CWAgent` namespace. Its label uses the account name to help identify the source account.

- **`AnsibleSSMStagingBucket`:** Named `ansible-ssm-staging-<account-id>-<region>`. It enables AES256 server-side encryption, blocks public access and expires abandoned objects after one day. Bucket versioning is not enabled.
- **`StagingBucketPolicy`:** Denies S3 access when `aws:SecureTransport` is false, enforcing TLS.

OAM provides visibility into source metrics. Metrics stay in their workload accounts, and links must use a matching regional sink. Ansible module transfers are temporary; the bucket does not store disk measurements.

Outputs expose the link ARN and staging bucket name.

3.5 `controller_policy.example.json`

Source: controller policy example.

This is an example policy to attach to the existing central controller role. It permits `sts:AssumeRole` on the listed member execution roles and `sns:Publish` on the listed central regional topics.

The controller needs permission to assume a role, and the member role must trust the controller. Both sides are required. Replace the example account and topic ARNs when onboarding actual accounts.

Administrator credentials provision CloudFormation stacks and attach policies. The recurring runtime jobs use the narrower controller and member roles.

3.6 How Ansible reaches a VM

The controller's AWS profile requests temporary member-role credentials through STS. Ansible discovers instance IDs and opens a Session Manager connection to the selected instance. Its SSM connection plugin stages module files in S3 and creates signed transfer URLs. The VM uses those URLs during module execution.

The controller has staging-bucket IAM permissions. The VM does not need a separate staging-bucket policy on its own role. Its SSM Agent must already be installed, online and permitted to communicate with AWS before Ansible can use this connection.

Linux VMs need Python 3.8 or newer and `curl`. The controller needs Python 3.11 or newer, Ansible, the AWS CLI and Session Manager plugin. Network connectivity must reach the required AWS services and installation sources; details are in the README.

3.7 CloudFormation syntax

Expression	What it does
`!Ref`	Returns a parameter value or a resource's reference value
`!GetAtt Resource.Attribute`	Reads a resource attribute, such as its ARN
`!Sub`	Replaces `${...}` placeholders in a string
`!Split`	Turns a comma-separated string into a list
`!If`	Chooses between two values using a named condition
`!Equals` / `!Not`	Builds the optional-account condition
`AWS::NoValue`	Removes a conditional property or list item
`AWS::AccountId`, `AWS::Region`, `AWS::Partition`	AWS-provided values for the stack being deployed

`AWSTemplateFormatVersion` identifies the template format. `Parameters` are inputs, `Resources` are what the stack creates, and `Outputs` expose values needed by later steps. IAM policy `Version: '2012-10-17'` identifies the policy language version.

4. Target configuration

Source: targets.yml.

Field	Meaning
`monitoring_account_id`	Central account ID used to validate SNS notification identity
`controller_profile`	Optional central AWS profile for coverage publishing; `null` uses the normal credential chain
`targets`	List of workload account/region combinations
Target `name`	Unique readable identifier, also used as a local state-directory name
Target `account_id`	Account expected from STS identity verification
Target `profile`	Local AWS profile that assumes that account's execution role
Target `region`	Region used for discovery, metrics, alarms and SSM
Target `sns_topic_arn`	Central topic in that same region

An account with VMs in two regions needs two target entries. They may use the same assumed-role profile, but the account/region pair must be unique. Target names must also be unique.

The AWS profiles live in the controller user's AWS configuration, outside this repository. The README shows profiles using `credential_source = Ec2InstanceMetadata`, so the controller's EC2 role supplies the starting credentials without stored access keys.

5. The deployment and audit runner

Source: scripts/run_monitoring.py.

5.1 Imports and constants

Python standard-library imports handle command-line arguments, paths, timestamps, JSON, hashing, environment variables, temporary files and subprocess execution. `boto3` is the AWS SDK, `botocore` supplies SDK configuration/errors, and `yaml` reads configuration files. `audit_target` comes from the sibling auditor module.

`ROOT` finds the repository from the script's own location, so default paths do not depend on the user's current working directory. `AWS_CONFIG` selects standard SDK retries with a bounded retry setting.

5.2 Functions

Function	Responsibility
`read_yaml(path)`	Read a UTF-8 YAML file using `safe_load`
`load_config(path)`	Validate the target configuration before work starts
`write_json(path, data)`	Create parent directories and replace a complete JSON file
`controller_lock(state_root)`	Prevent overlapping jobs using the same state directory
`check_identity(session, expected_account)`	Verify the AWS account through STS
`deploy_target(target, state_dir)`	Prepare one target's inventory/variables and run Ansible
`notify_changes(session, target, result, state_dir)`	Publish changed coverage results and record notification history
`main(argv=None)`	Parse commands, select targets, coordinate execution and return an exit code

5.3 Configuration validation

`load_config` requires a mapping containing a nonempty target list. Each target must have nonempty string fields for its name, account, profile, region and topic.

Regular expressions check account-ID, target-name and region formats. Target names allow lowercase letters, digits and hyphens, with at most 63 characters. Restricting names also prevents paths such as `../outside` from becoming state directories.

The expected topic ARN is constructed from the central account, target region and fixed topic name `enterprise-disk-alerts`. The supplied ARN must match it. Two sets detect duplicate names and duplicate account/region pairs.

5.4 JSON writes and the lock

`write_json` writes to a sibling `.tmp` file, then replaces the destination. Readers therefore see the complete old file or complete new file rather than a partially written JSON document.

`controller_lock` is a context manager: entering its `with` block acquires the lock, and leaving releases it. Linux `fcntl.flock` uses exclusive, nonblocking locking. An overlapping run fails immediately instead of waiting or changing shared state concurrently.

The lock covers the whole deployment/audit run for one state root. It does not stop agents or CloudWatch alarms. Different state roots have independent locks.

5.5 Deployment preparation and execution

`deploy_target` first records a deployment start time. It preserves the previous `first_started_at`, which prevents repeated deployments from extending initial enrollment grace indefinitely.

It then reads the base EC2 inventory and replaces its profile and region list with the selected target. A temporary directory holds this generated inventory and an extra-variable JSON file. The inventory filename ends in `.aws_ec2.yml` for the EC2 inventory plugin.

Extra variables supply the target region/profile, SNS topic, staging bucket and absolute manifest directory. `target_account_id` and `deployment_started_at` are also passed as context values; the current role does not consume them. Manifest timestamps are collected per host at discovery time.

The child environment removes inherited direct AWS keys/session tokens and `AWS_DEFAULT_PROFILE`, then sets the target profile and region explicitly. It also sets AWS retry environment settings and the absolute Ansible configuration path. This keeps the child invocation tied to the selected profile.

`subprocess.run` starts `ansible-playbook` with the generated inventory and `--extra-vars @<file>`. Arguments are passed as a list, rather than a shell command string. A nonzero playbook exit becomes a deployment failure.

Inside the protected Ansible execution block, failures and interruptions set deployment status to `failed`. A `finally` block records the completion time. A completed playbook sets `success`, but a later audit is still needed to verify telemetry and alarms.

5.6 Notification deduplication

`notify_changes` sorts findings and combines them with the result status. SHA256 turns that content into a fingerprint. If it matches the saved fingerprint, the function sends no new notification.

The fingerprint excludes timestamps and summary counts, so an unchanged problem does not create a new email merely because another audit ran. The initial `ok` or `pending` result is recorded quietly. Changed problems and subsequent recovery can produce notifications.

The SNS request uses the target's central regional topic. Subject and message lengths are bounded. Notification state is saved after successful publication; a failed publish leaves the previous fingerprint in place so a later run retries.

5.7 Commands and exit behavior

```
python scripts/run_monitoring.py list
python scripts/run_monitoring.py deploy --target workload-a-us-east-1
python scripts/run_monitoring.py audit --notify
```

Argument	Behavior
`list`	Display configuration without AWS calls; works on any OS
`deploy`	Run Ansible for selected targets; requires Linux
`audit`	Verify coverage for selected targets; requires Linux
`--targets PATH`	Read another target YAML file
`--target NAME`	Select a target; repeat the flag to select several
`--state-dir PATH`	Use another state root
`--notify`	Publish changed audit results; valid only with `audit`

`main` validates configuration, selects targets, obtains the lock and creates AWS sessions. Each target's identity is checked before deployment or auditing. A failed target does not prevent the remaining targets from being processed.

With `--notify`, the publisher identity must match the central account. A publisher failure disables notifications for that run, while target audits continue and the final exit code indicates failure.

Audits return zero only when all selected targets are verified `ok` and notification delivery, if requested, succeeds. Gaps, errors and pending coverage return nonzero. Deployment returning zero means the playbook completed. Argument errors can have a separate nonzero parser exit code.

The `if __name__ == '__main__'` block runs the command when the file is executed directly. It prints handled top-level errors to standard error and exits with a failure code.

6. Ansible configuration, inventory and playbook

6.1 `ansible.cfg`

Source: Ansible configuration.

Setting	Purpose
`inventory`	Default EC2 inventory file; the runner supplies a generated replacement
`roles_path`	Directory where Ansible finds the project role
`forks = 25`	Up to 25 concurrent task workers
`retry_files_enabled = False`	Avoid creating separate `.retry` files
`stdout_callback = default`	Use standard Ansible task output
`enable_plugins`	Permit static inventories and the AWS EC2 inventory plugin

The runner sets `ANSIBLE_CONFIG` explicitly so Ansible uses this file. Static inventory support also allows the README's offline syntax check with `localhost,`.

6.2 `inventory/aws_ec2.yml`

Source: dynamic inventory.

`plugin: amazon.aws.aws_ec2` asks Ansible to query EC2. The filter includes running instances, and `hostnames: instance-id` uses IDs such as `i-...` as inventory names.

`compose` creates connection variables:

- `ansible_host` is the instance ID, which SSM uses to select the VM.
- `ansible_connection` selects `amazon.aws.aws_ssm`.
- `ansible_aws_ssm_region` comes from the instance's placement region.

The nested quotes around the connection plugin make its name a literal string in the inventory expression. `groups` places Windows instances in `windows_instances`; other instances enter `linux_instances`, where unsupported platforms fail the role's platform checks.

The runner supplies the profile and exactly one region per invocation. There is no tag-based enrollment filter: all running instances in a configured target are discovered.

6.3 `inventory/group_vars/all.yml`

Source: shared settings.

Both the playbook and Python auditor read these settings so they agree on expected metrics and alarm behavior.

Setting	Default	Meaning
`cw_namespace`	`CWAgent`	Metric namespace
`cw_metric_collection_interval_seconds`	60	Agent collection interval
`linux_disk_metric_name`	`disk_used_percent`	Published Linux metric name
`windows_disk_metric_name`	`disk_free_percent`	Published Windows metric name
`disk_warning_threshold`	80	Warning used-percent threshold
`disk_critical_threshold`	90	Critical used-percent threshold
`alarm_period_seconds`	60	Length of an alarm aggregation period
`alarm_evaluation_periods`	3	Number of evaluated periods
`alarm_datapoints_to_alarm`	3	Breaching periods required
`coverage_max_age_minutes`	5	Maximum age of acceptable metric data
`coverage_inventory_max_age_minutes`	30	Maximum age of discovery and successful deployment state
`enrollment_grace_minutes`	15	Bounded initial/recent-launch grace period
`linux_ignored_filesystems`	List	Filesystem types omitted from discovery and collection

The exclusion list contains kernel interfaces such as `proc`, `sysfs` and `cgroup`, memory-backed filesystems such as `tmpfs` and `ramfs`, and types such as `overlay` and `squashfs`. These are outside the selected persistent-filesystem monitoring scope.

Windows alarm thresholds are calculated as `100 - used_threshold`, giving 20% and 10% free. Changing namespace or metric names also requires updating OAM filters and dashboard searches. Dashboard annotations need to match threshold changes.

6.4 `playbook_deploy_monitoring.yml`

Source: deployment playbook.

The playbook has one Linux play and one Windows play. Each gathers facts and loads the shared settings explicitly with `vars_files`. Both use the `cloudwatch_disk_agent` role.

Linux tasks use privilege escalation with `become: true` and a `sh` shell. Windows tasks use PowerShell with `become: false`. The SSM profile is set from the selected target's profile.

`serial: 25` handles hosts in batches of 25. This bounds the hosts in a play batch; `forks: 25` bounds concurrent workers within that batch. Host facts provide information such as the OS, architecture and

mounted filesystems.

7. The CloudWatch Agent role

7.1 `vars/main.yml`

Source: role variables.

This file defines package sources, distribution/architecture mappings, signing-key information, control-script paths, configuration paths and service names.

The Linux mappings translate fact names into download-path names: `Amazon` to `amazon_linux`, `RedHat` to `redhat`, `x86_64` to `amd64`, and `aarch64` to `arm64`. Ubuntu and Debian have their own download paths.

Linux installation uses `/var/tmp/cloudwatch-agent-install` for package verification and the agent's standard `/opt/aws/amazon-cloudwatch-agent` paths. Windows downloads the MSI into `C:\Windows\Temp`, uses the control script under `C:\Program Files\Amazon\AmazonCloudWatchAgent`, and writes configuration under `C:\ProgramData\Amazon\AmazonCloudWatchAgent`.

The Linux signing-key fingerprint is pinned in this file. Initial installation downloads the current `latest` package from the official package source; recurring deployment does not upgrade an installed agent.

7.2 `tasks/main.yml`

Source: role entry point.

This file defines the order: discover disks, run the matching OS installation/configuration tasks, then create alarms. `include_tasks` loads the relevant task file at runtime, and `when` selects the OS branch.

Alarm tasks are delegated to `localhost` with a local connection and no privilege escalation. They call AWS APIs from the controller. Explicit local shell and Python settings prevent a Windows host's PowerShell settings from affecting those delegated tasks.

7.3 `tasks/discover.yml`

Source: filesystem discovery.

Linux discovery starts with `ansible_facts.mounts`. The expression:

```
ansible_facts.mounts
  | rejectattr('fstype', 'in', linux_ignored_filesystems)
  | map(attribute='mount') | unique | sort
```

removes excluded filesystem types, extracts mount paths, removes duplicates and sorts them. For example, eligible mounts `/` and `/data` become a list of those two strings.

Windows discovery queries `Win32_LogicalDisk` with `DriveType = 3` for fixed drives, extracts `DeviceID` values and sorts them uniquely. `windows_drives.output` becomes the filesystem list. `$Ansible.Changed = $false` identifies this query as a read operation. `SupportsShouldProcess` allows this discovery script to run in check mode, and `error_action: stop` makes PowerShell errors fail the task.

An assertion requires at least one filesystem. The final task writes a controller-side manifest with instance ID, platform, collection timestamp and filesystem list. The timestamp is taken from the controller in UTC for each host. File mode `0600` allows only the owner to read/write it.

The manifest is saved before installation. If the agent cannot be installed, its disks remain in the expected set for later auditing.

7.4 `tasks/linux.yml`

Source: Linux tasks.

The tasks first assert that the VM is Linux and that its distribution and architecture appear in the supported mappings. A `stat` task checks whether the agent control script already exists.

If absent, and outside check mode, the installation block:

1. Chooses a `.deb` package for Debian-family hosts or `.rpm` for Red Hat-family hosts.
2. Installs `gnupg` or `gnupg2` for signature verification.
3. Creates private installer and keyring directories owned by root with mode `0700`.
4. Downloads the package, detached `.sig` signature and AWS public signing key, using mode `0600`.
5. Imports the key into the private keyring.
6. Runs `gpg --verify` and requires both successful exit and a `VALIDSIG` record containing the pinned fingerprint.
7. Installs the verified package through `rpm -U` or Ansible's `apt` module.

`argv` supplies command arguments separately. `changed_when: false` keeps verification commands from appearing as configuration changes. `failed_when` combines the exit-code and trusted-fingerprint checks. The RPM task's `creates` setting prevents execution when the control script already exists.

After installation, the role creates the configuration directory, renders the Linux template and notifies the configuration handler if the file changes. `flush_handlers` applies pending configuration before the service check. The service is started and enabled across reboots.

Configuration mode `0644` permits reading the file; it contains settings rather than stored credentials. Existing installations are preserved, so this presence check is not a version/health check. Package upgrades are a separate maintenance operation.

7.5 `tasks/windows.yml`

Source: Windows tasks.

An assertion requires a 64-bit Windows host. `win_stat` checks for the agent control script.

When absent, the role downloads the MSI and uses `Get-AuthenticodeSignature` to require a valid signature with publisher `Amazon Web Services, Inc.`. An invalid signature or unexpected publisher throws an error. `win_package` installs the verified MSI.

The remaining tasks create the configuration directory, render the Windows template, apply pending handlers and keep `AmazonCloudWatchAgent` started with automatic startup. Existing installations are preserved.

7.6 `handlers/main.yml`

Source: configuration handlers.

A handler runs when a task notifies it after a change. Installation and configuration-file changes notify the matching OS handler. Multiple notifications before a flush are combined into one handler execution for that host.

The Linux handler runs the agent control command with:

Argument	Meaning
`-a fetch-config`	Load the supplied configuration
`-m ec2`	Use EC2 mode
`-s`	Start/restart the agent with the configuration
`-c file:<path>`	Read configuration from the local file

The Windows handler invokes `powershell.exe` with the agent control script and equivalent arguments. It checks `$LASTEXITCODE` and throws if configuration failed.

Handlers skip execution in check mode. An unchanged configuration does not trigger a handler, while the service task still ensures the agent is started.

8. Metrics, templates and alarms

8.1 Why metric dimensions matter

CloudWatch identifies a metric series using its namespace, name and complete set of dimensions. These must agree in the agent configuration, alarm configuration and audit query.

Platform	Namespace and metric	Exact dimensions
Linux	`CWAgent / disk_used_percent`	`InstanceId`, `path`
Windows	`CWAgent / disk_free_percent`	`InstanceId`, `instance`, `objectname`

Linux `path` may be `/data`. Windows `instance` may be `C:`, and `objectname` is `LogicalDisk`. A query using a different dimension set asks for a different series and may return no data.

8.2 Linux agent template

Source: Linux template.

`agent.metrics_collection_interval` uses the shared 60-second default. `run_as_user: root` supplies access to filesystem information. `metrics.namespace` selects `CWAgent`, and `append_dimensions` adds the EC2 instance ID.

Under `metrics_collected.disk`:

- `measurement` collects `used_percent`, publishes it using the configured Linux metric name and sets unit `Percent`.
- `resources: ["*"]` selects eligible mounted filesystems rather than a hardcoded mount list.
- `ignore_file_system_types` uses the same exclusions as Ansible discovery.
- `aggregation_dimensions` publishes series using only `InstanceId` and `path`.
- `drop_original_metrics` suppresses the original `disk_used_percent` variants containing additional disk labels.

This dimension aggregation creates the per-mount series used by the alarms. It does not calculate a single usage percentage for an entire VM or account. The suppression setting contains the collector's original metric name.

`to_json` encodes inserted strings and lists as valid JSON, including quoting and escaping.

8.3 Windows agent template

Source: Windows template.

The template selects `LogicalDisk` performance counters and collects `% Free Space`. It renames the published metric to `disk_free_percent`, sets unit `Percent`, adds the instance ID and selects resources with `*`.

The native `instance` and `objectname` dimensions are retained. No Linux-style dimension aggregation is configured. Wildcard counters may also publish `_Total` or other series, but per-drive alarms use the independently discovered fixed drives.

8.4 `tasks/alarms.yml`

Source: alarm tasks.

Four tasks create Linux warning/critical alarms and Windows warning/critical alarms. Each loops over the discovered filesystem list and runs only for the relevant OS. Calls use the target's explicit profile and region on the controller.

Alarm names contain severity, instance ID and the first 12 characters of a SHA1 hash of the filesystem name. Hashing keeps paths containing slashes or spaces out of the generated suffix while making repeated deployments use the same name. This hash identifies a filesystem; it is not used for authentication.

The default configuration is:

Property	Linux	Windows
Warning condition	Used percentage `≥ 80`	Free percentage `≤ 20`
Critical condition	Used percentage `≥ 90`	Free percentage `≤ 10`
Statistic	Average	Average
Period	60 seconds	60 seconds
Evaluation periods / required breaches	3 / 3	3 / 3
Missing-data policy	`missing`	`missing`
Alarm and OK actions	Central regional SNS topic	Central regional SNS topic

Three breaching one-minute periods are required; exact notification timing can also include publication and evaluation delays. The warning and critical alarms are independent, so high usage can activate both. They use the same email channel, with severity in their names and descriptions.

`TreatMissingData: missing` prevents absent measurements from being treated as a capacity threshold breach. Monitoring coverage is checked separately by the auditor.

The last task restores disabled actions. The pinned alarm module does not restore this property when its other settings already match. Registered task results are combined, results without `actions_enabled` are filtered out, and only results where it is false invoke `aws cloudwatch enable-alarm-actions`.

9. The coverage auditor

Source: scripts/coverage_reconciler.py.

The auditor reads AWS and controller state. It does not configure VMs, create alarms, delete alarms or write controller state. The runner handles optional notifications separately.

9.1 Helper functions

Function	Responsibility
`utc_time(value)`	Convert an SDK datetime or ISO timestamp to timezone-aware UTC
`read_json(path)`	Load a local JSON file
`disk_metric( … )`	Construct the expected metric identity for one filesystem
`metric_key(metric)`	Build a comparable key from namespace, name and sorted dimensions
`alarm_matches( … )`	Check an alarm's configuration against the required severity and OS
`recent_metrics( … )`	Query metric freshness in bounded batches and follow result pages
`audit_target( … )`	Combine deployment, discovery, SSM, alarm and metric checks

`metric_key` uses a tuple containing sorted dimension pairs so differing dimension order does not change identity. Sets are used to track online instances and recent query IDs efficiently.

9.2 Checking deployment state

`audit_target` reads `deployment.json` and validates its timestamps and status. A recent running deployment is pending; an overlong running deployment or failed deployment is an error. A successful deployment becomes stale after the configured discovery-age limit.

Missing, malformed or future-dated state produces an error. A previous manifest alone cannot prove that current discovery completed successfully.

9.3 Independently discovering running VMs

The auditor uses paginated EC2 discovery filtered to running instances. It also checks the state field in returned records. Stopped instances are excluded from expected coverage.

It reads all SSM instance-information pages to identify instances with `PingStatus: Online`, and all `Disk-*` alarm pages. Alarms are grouped by metric identity.

The auditor builds expectations from EC2 instances and manifests rather than from the list of metrics that happen to exist. Therefore, an expected disk that never emitted a metric remains detectable.

9.4 Enrollment grace and manifest validation

For each instance, grace starts at the later of its EC2 launch time and the target's preserved first-deployment time. Missing coverage within 15 minutes is pending; after that it is a gap. Refreshing a manifest does not reset grace.

The manifest must match the instance ID and platform, have a recent timestamp, and contain a nonempty list of unique, nonempty filesystem strings. A stale or malformed manifest is an error rather than accepted coverage. An absent manifest is pending or a gap depending on grace.

An offline SSM agent is also pending or a gap. Metrics may still exist, but configuration access cannot be verified.

9.5 Verifying both alarms

For each expected filesystem, the auditor finds alarms for its exact metric identity. It requires correctly configured warning and critical conditions.

`alarm_matches` checks threshold, comparison operator, Average statistic, period, evaluation periods, required datapoints, missing-data policy, enabled actions, and the expected SNS topic in both alarm and recovery actions. Namespace, name and dimensions are matched through `metric_key`.

Finding an alarm by prefix alone is insufficient. The auditor does not require an alarm to be in `OK`: an actively alarming disk can still have verified monitoring coverage. Alarm delivery and disk capacity are handled by CloudWatch.

9.6 Checking recent metric data

Each filesystem gets a query ID such as `m0` and an exact metric query. `recent_metrics` sends at most 500 queries per call and follows `NextToken` pages.

The lookback interval is at least 15 minutes, but a datapoint must fall within the configured freshness window, no later than the audit time. With defaults, this means at least one finite datapoint in the last five minutes. Zero is a valid reading; missing data and non-finite values do not establish freshness.

The function checks final result status and API messages. Missing results, errors and incomplete responses prevent the audit from reporting verified coverage. A fresh datapoint alone does not hide a query error.

9.7 Obsolete alarm candidates

The auditor can list project disk alarms for filesystems no longer present in a fresh manifest of a running VM. It only uses verified discovery to identify these candidates.

Candidates are informational and are not automatically deleted or used alone to fail the audit. Unknown, stopped or retired hosts need a separate manual review because missing discovery does not safely establish that an alarm should be removed.

9.8 Results

Status	Meaning
`ok`	Every expected filesystem on running instances passed the coverage checks
`pending`	Initial/recent-launch coverage is still unverified within grace
`gap`	Expected monitoring or configuration access is missing/misconfigured beyond grace
`error`	State or API failures prevent reliable verification

Status priority is `error`, then `gap`, then `pending`, then `ok`. All findings are retained and sorted. Stable finding text allows notification deduplication.

The returned dictionary contains `status`, `findings`, `summary` and `stale_alarms`. A target with zero running instances can pass if its deployment state is valid. The summary's counts let the operator see that no running VMs were checked.

The outer exception handler records an audit error instead of allowing an API failure to appear healthy. The runner can then continue to other targets.

10. Generated controller state

The default state root is `.monitoring-state`. Each target has its own directory:

```
.monitoring-state/
├── controller.lock
└── workload-a-us-east-1/
    ├── deployment.json
    ├── i-<instance-id>.json
    └── notification.json
```

File	Writer	Purpose
`controller.lock`	Runner	Shared Linux lock for the state root
`deployment.json`	Runner	First enrollment time and latest deployment start, finish and status
`<instance-id>.json`	Ansible discovery task	Instance identity, OS and expected filesystems
`notification.json`	Runner	Last recorded notification fingerprint and status

The following manifest is illustrative, not a captured deployment result:

```
{
  "instance_id": "i-example",
  "platform": "linux",
  "collected_at": "2026-10-11T12:00:00Z",
  "filesystems": ["/", "/data"]
}
```

`first_started_at` in deployment state is preserved across runs. `started_at` and `finished_at` describe the latest deployment. The filesystem `collected_at` timestamp describes discovery for that particular host.

State is operational data and should remain available to recurring jobs. Keep jobs under the same controller user with consistent AWS configuration and state paths. Deleting state loses expectations, first-enrollment history and notification deduplication history; it does not delete AWS alarms or metrics.

The runner's generated inventory and extra-variable files live in a temporary directory and are removed after the invocation. They are distinct from persistent coverage state.

11. Dependencies and excluded files

11.1 Python requirements

Source: requirements.txt and requirements-dev.txt.

`requirements.txt` contains `boto3≥1.35,<2` and `PyYAML≥6.0,<7`. Boto3 supplies AWS sessions/clients; PyYAML supplies YAML parsing.

`requirements-dev.txt` includes the runtime file through `-r requirements.txt`, then adds `ansible-core≥2.18,<2.19` and `cfn-lint≥1.40,<2`. This supports the controller and offline validation. The ranges permit updates within those bounds rather than pinning every Python dependency exactly.

11.2 Ansible collections

Source: ansible/requirements.yml.

`amazon.aws` is pinned to `11.4.0` for inventory, SSM connections and CloudWatch alarm management. `ansible.windows` is pinned to `3.8.0` for Windows discovery, installation, files, templates and services. Python installation alone does not install these collections; use `ansible-galaxy collection install` as shown in the README.

The AWS CLI and Session Manager plugin are separate controller prerequisites. The CLI is also used by the disabled-alarm-action repair task.

11.3 `.gitignore`

Source: .gitignore.

Pattern	What it excludes
`__pycache__/`, `*.py[cod]`	Generated Python bytecode
`.venv/`, `.venv-linux/`	Local Python environments
`.tools/`	Downloaded tools and local validation material
`.monitoring-state/`	Controller operational state
`.aws/`	Local AWS configuration/credential material
`.env`	Local environment settings

These exclusions keep generated and local configuration files out of source control. They do not remove files already tracked by Git or guarantee that every possible secret is excluded. Review actual files before publication.

12. Tests and verification

12.1 `tests/test_runner.py`

Source: runner tests.

The tests use temporary directories and mocked AWS sessions/subprocesses. They do not assume roles or deploy anything.

The seven test methods cover:

Case	What it protects
Duplicate targets and unsafe names	State-path safety and unique account/region selection
Wrong account/region in topic ARN	Correct central notification destination
Identity mismatch	Account verification before deployment
Generated deployment inputs	Profile/region isolation, cleared inherited credentials and preserved first enrollment
Failed playbook	Failed deployment state and completion timestamp
Repeated coverage findings and recovery	Notification deduplication and recovery publishing
Failed SNS publish	Retry without prematurely recording delivery

`setUp` creates fresh temporary state per test; cleanup removes it afterward. `patch` temporarily replaces real functions/environment values. The deployment test reads the temporary files while the mocked subprocess is invoked, verifying the inputs Ansible would receive.

12.2 `tests/test_coverage_reconciler.py`

Source: coverage tests.

The test fixture supplies a fixed UTC time, a successful deployment record, a Linux VM with two filesystems, online SSM status, alarms and mocked metrics. Individual tests change one part of that fixture.

The twelve methods cover:

- Fresh metrics and both alarms for every expected disk.
- Valid zero readings and rejection of stale/future timestamps.
- Incorrect alarm actions, thresholds, dimensions and evaluation settings.
- Multiple EC2, SSM and alarm pages, including Windows metric dimensions.
- Missing/stale manifests and failed deployment state.
- A newly expected disk that never emitted metrics.
- Grace expiry without resetting grace on manifest refresh.
- Offline SSM and exclusion of stopped instances.
- API failure and incomplete metric result statuses.
- Metric pagination and the 500-query batch limit, using 501 queries.
- Stable findings for an unchanged problem.
- Conservative obsolete-alarm candidates based on fresh discovery.

Mocked clients return local data. These tests check decision logic, not live AWS authorization, SSM connectivity or actual agent publication.

12.3 `tests/simulate_disk_pressure.sh`

Source: controlled disk-pressure script.

This is a live pilot aid for a disposable Linux VM and a separately mounted test filesystem. It writes data; it is not part of recurring monitoring.

The script:

1. Accepts a mount path and a target of 85 or 95 percent; 85 is the default.
2. Resolves the path and refuses root and common system paths.
3. Requires a mount point on a different device from root and an `ext4` or `xfs` filesystem.
4. Reads capacity/usage/free-space figures and requires a filesystem no larger than 1 GiB.
5. Calculates the bytes needed for the requested target, rounds allocation to whole MiB and preserves at least 16 MiB free.
6. Creates a uniquely named payload file and installs cleanup traps before allocation.
7. Writes zeros with `dd`, flushes the data, and waits for the operator to verify metric, alarm and SNS delivery.
8. Removes the payload and asks the operator to verify recovery.

`set -euo pipefail` stops on command failures, unset variables and failed pipeline elements. `mktemp` creates the unique file. `trap` handles exit and interruption cleanup. `conv=fsync` flushes written data before the allocation command finishes.

Reserved blocks and filesystem accounting can affect the observed percentage. The target is approximate, and the script refuses a target it cannot reach while retaining its reserve. Follow the README's disposable-test setup instructions.

12.4 What the other checks establish

`cfn-lint` checks CloudFormation templates against resource schemas and additional rules. Ansible syntax checks verify playbook/task parsing; dynamically included files may need separate checks. Rendering the two templates as JSON checks their generated configuration structure. `bash -n` checks shell syntax without allocating data.

Offline checks do not prove a successful AWS deployment. A live pilot must verify real Linux and Windows execution, metric dimensions, OAM visibility, notifications, repeat-deployment behavior and failure/recovery scenarios. The README provides the pilot procedure.

13. Complete flows and operational boundaries

13.1 First setup

1. Provide the existing central controller role and install controller dependencies.
2. Deploy a central monitoring stack in every monitored region.
3. Deploy member IAM once per workload account.

4. Deploy each member regional stack using the matching central sink ARN.
5. Bootstrap VM SSM access, permissions, prerequisites and connectivity.
6. Configure the controller policy, assumed-role profiles and target entries.
7. Confirm email subscriptions, then run deployment and audit.

CloudFormation does not attach the instance profile to existing VMs, install their SSM Agent or create the controller machine. These are explicit prerequisites.

13.2 A normal deployment

The runner acquires its lock and verifies one target's identity. It prepares inventory and variables, then Ansible discovers running VMs. Each host's filesystems are recorded, its agent is installed/configured as needed, and its alarms are created or repaired. The runner records deployment completion and proceeds to the next target.

13.3 A disk running low

The agent publishes a disk reading. CloudWatch evaluates the workload-account alarm, and a sustained threshold breach changes alarm state. CloudWatch publishes to the central regional SNS topic, which emails its confirmed subscriber. Recovery produces an OK notification.

Neither the controller nor OAM is required for this alarm evaluation path. OAM supplies the central dashboard view.

13.4 A monitoring failure

If an agent stops publishing, the next audits stop accepting its old datapoints after the freshness window. The auditor reports a coverage finding, and `audit --notify` publishes the changed result. Unchanged findings are suppressed; later verified recovery can produce another notification.

With five-minute metric freshness and a five-minute audit schedule, detection is normally within roughly ten minutes, subject to job availability and API/runtime delays.

13.5 A new VM or disk

A bootstrapped new VM is picked up by the next deployment's running-instance inventory. Its disks are discovered and receive configuration/alarms. A new disk on an existing VM may begin publishing through wildcard collection before deployment; the next discovery adds it to expectations and creates its two alarms.

Before discovery succeeds, the controller cannot claim to know every newly attached filesystem. Failed or stale discovery is reported so that this uncertainty remains visible.

13.6 Scheduling and growth

The README's cron examples deploy every 15 minutes and audit every five minutes. They are examples to install through the existing operations process; this repository does not register jobs automatically.

The 30-minute discovery limit must exceed deployment interval plus maximum run time. The shared lock causes overlapping jobs to fail instead of waiting, so long deployments can skip scheduled audits. Monitor

job failures and adjust schedules, freshness settings or target distribution as the fleet grows.

StackSets can distribute account and regional baselines to Organization accounts. The project does not automatically discover every new AWS account or create its controller profile. Account onboarding still includes permissions, profiles, targets and VM bootstrap.

13.7 Scope and limits

- Supported hosts are Amazon Linux, RHEL, Ubuntu and Debian on x86_64/ARM64, and 64-bit Windows Server, with the documented prerequisites.
- The implementation measures filesystem capacity. It does not measure inode exhaustion, resize disks or delete workload data.
- Windows discovery covers fixed logical drives; directory-only mount points and clustered layouts require an extension.
- Central metric visibility is regional, and dashboard searches have series limits. Alarm and audit coverage are separate from how many lines fit on a dashboard.
- Threshold, namespace and metric-name changes may require corresponding dashboard/OAM changes.
- Installed agent presence is checked, but upgrades are a separate maintenance process.
- Retired instances and obsolete alarms need manual review. Runtime jobs never delete alarms automatically.
- Controller availability affects enrollment and coverage notifications. Existing agent publication and CloudWatch capacity alarms continue independently.

13.8 Cost model

Costs depend on published filesystem series, two alarms per eligible filesystem, dashboards, SNS, S3, controller hosting and network access. For example, 1,000 VMs averaging two monitored filesystems imply about 2,000 capacity series and 4,000 alarms, before any extra Windows series. The README links current pricing sources.

14. Suggested reading order

Start with `targets.yml` and the shared settings to understand what is selected and measured. Next read the inventory, playbook and role task files to follow deployment on one host. Read the two templates and alarm tasks together to understand exact metric identity.

Then read `run_monitoring.py` for account/region coordination and `coverage_reconciler.py` for coverage decisions. Finish with the CloudFormation policies and tests to understand access boundaries and failure cases. Use the README for commands when moving from understanding the files to an actual deployment.